📐 Architecture & System Design

C-Square × GIROPie Non-ERP Buyer Collection Platform

🎯 Objective

Design a scalable, secure, and fintech-ready architecture where:

- Seller uses C-Square ERP
- Buyer does not use ERP
- Buyer receives payment request link
- Buyer authorizes eMandate
- Payment executes automatically on due date
- ERP remains synchronized
- Platform acts as orchestration layer

🌐 High-Level Architecture

Seller (C-Square ERP)

↓

ERP Integration Layer

↓

Your Platform (Workflow Engine)

↓

Buyer Portal

↓

Payment Partner (Easebuzz)

↓

NPCI / Banking Network

↓

Settlement + Callbacks

↓

Your Platform

↓

ERP Update Layer

🧩 Core Architecture Layers

The system should be divided into clear layers.

1️⃣ ERP Integration Layer

Responsibility

Acts as connector between C-Square ERP and platform.

Responsibilities

- Receive invoice data from ERP
- Push mandate updates back
- Push payment status back
- Sync invoice lifecycle

Example API

POST /invoice/create

POST /invoice/cancel

POST /invoice/update

2️⃣ Workflow Orchestration Layer

Most Important Layer

This becomes the “brain” of the platform.

Responsibilities

- Create buyer relationship
- Create request lifecycle
- Generate secure links
- Control mandate lifecycle
- Decide when debit should happen
- Handle retry logic
- Manage disputes

This Layer Controls

Invoice → Buyer → Mandate → Payment → ERP Update

3️⃣ Buyer Identity Layer

Purpose

Recognize buyers across sellers.

Buyer Identity Keys

Recommended:

- Mobile Number
- GSTIN
- PAN
- Business Name

Responsibilities

- Create buyer profile
- Match existing buyer
- Group invoices by buyer
- Show unified dashboard

4️⃣ Buyer Portal Layer

UI Layer

Buyer-facing website.

Buyer Features

- Login/signup
- View pending invoices
- Seller grouping
- Approve mandate
- Request extension

- Split payment
- View payment history

5️⃣ Mandate Service

Purpose

Manage eMandate lifecycle.

Responsibilities

- Create mandate request
- Send to Easebuzz
- Store UMRN
- Check mandate status
- Renew expired mandate

Mandate States

Pending

Approved

Rejected

Expired

Paused

Cancelled

6️⃣ Payment Scheduler Service

Purpose

Decides when debit should happen.

Responsibilities

Daily background job.

Checks:

- invoice due date
- extension status
- split payment logic

- dispute hold
- mandate active

Example

Run every day at midnight

Check all due invoices

Trigger debit where applicable

7️⃣ Payment Execution Layer

Purpose

Communicates with payment partner.

Responsibilities

- Trigger debit request
- Retry failed debit
- Receive callback
- Store UTR

Partner

entity["company","Easebuzz","Indian payment gateway and recurring payment platform"]

8️⃣ Settlement & Reconciliation Layer

Purpose

Keep financial records correct.

Responsibilities

- Match UTR to invoice
- Update payment status
- Detect duplicate debit
- Calculate outstanding balance

9️⃣ ERP Sync Layer

Purpose

Push updates back to ERP.

Responsibilities

Send to C-Square:

- mandate approved
- payment success
- UTR
- failed debit
- partial payment

🔄 Core System Flow

Invoice Created

↓

ERP Sends Data

↓

Platform Creates Buyer Relationship

↓

Secure Link Generated

↓

Buyer Opens Portal

↓

Buyer Login

↓

Buyer Approves eMandate

↓

Easebuzz + NPCI Validation

↓

UMRN Generated

↓

ERP Updated

↓

Scheduler Waits For Due Date

↓

Platform Triggers Debit

↓

Payment Success

↓

UTR Generated

↓

ERP Updated

🧠 **Recommended Microservice Design**

Instead of one large backend.

Use domain-based services.

Suggested Services

Invoice Service

Handles:

- invoice creation
- outstanding amount
- invoice status

Buyer Service

Handles:

- buyer profile
- identity mapping
- buyer dashboard

Mandate Service

Handles:

- eMandate lifecycle
- UMRN

Payment Service

Handles:

- debit execution
- payment retry

Scheduler Service

Handles:

- due-date checks
- recurring debit logic

ERP Sync Service

Handles:

- ERP callbacks
- synchronization

Notification Service

Handles:

- SMS
- WhatsApp
- Email

🗄️ Suggested Database Design

Buyer Table

Column	Description
buyer_id	Unique buyer
mobile	Primary identity

Column	Description
gstin	Business tax id
name	Buyer name

Seller Table

Column	Description
seller_id	ERP seller
company_name	Seller company

Invoice Table

Column	Description
invoice_id	ERP invoice
buyer_id	Owner buyer
seller_id	Owner seller
due_date	Due date
status	Payment state

Request Table

Tracks invitation.

| request_id | invoice_id | buyer_id |

Mandate Table

| mandate_id | buyer_id | seller_id | UMRN | status |

Payment Table

| payment_id | invoice_id | UTR | status |

Audit Log Table

Tracks every action.

⚠️ Critical Edge Cases

Buyer Already Exists

Reuse account.

Duplicate Invitation

Avoid duplicate request.

Mandate Already Active

Reuse existing mandate.

Partial Payment

Track remaining amount.

Buyer Has No Funds

Retry later.

Invoice Cancelled

Stop debit.

Buyer Dispute

Pause payment.

Failed ERP Callback

Retry sync.

Duplicate Debit

Prevent double charge.

NPCI Delay

Async polling needed.

🔐 Security Design

Must include:

- encrypted bank data
- OTP verification
- signed callback validation
- API authentication
- audit logging
- role-based access

📈 Scalability Design

Design for:

- multiple ERP clients
- multiple sellers
- millions of invoices
- recurring payment jobs
- async queue processing

Suggested Tech Stack

Backend

- Spring Boot
- Java
- REST APIs

Frontend

- React
- Material UI

Database

- PostgreSQL

Queue

- Kafka / RabbitMQ

Scheduler

- Quartz Scheduler

Cache

- Redis

Notifications

- SMS API
- WhatsApp API
- Email API

📌 **Final Architecture Understanding**

This platform is NOT a payment gateway.

It is:

ERP Extension Layer

+

Buyer Relationship Engine

+

Mandate Orchestration

+

Payment Lifecycle Manager

+

Fintech Workflow Infrastructure